

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FCCD – Software

Tarea:

Práctica Experimental Unidad 3

Materia:

Aplicaciones Web

Integrantes:

Álava Alvarado Jean Pierre

Rivera Suárez Jonathan Javier

Tejada Bajaña Luis Alejandro

Docente:

Ing. Guerrero Ulloa Gleiston Ciceron

Curso:

5to Software A

Quevedo – Los Ríos – Ecuador

19/06/2026

Índice

Índice	1
1. Introducción	3
2. Marco Teórico: Fundamentos de Arquitectura y Seguridad Web	3
2.1. Seguridad en Aplicaciones Web	3
2.1.1. Principios fundamentales de la seguridad	3
2.1.2. OWASP Top 10 y Principales Amenazas	4
2.1.3. Buenas prácticas de codificación segura	4
2.2. Objetos Integrados y Gestión de Estado en Aplicaciones Web	5
2.2.1. El objeto Request y Response	5
2.2.2. Gestión de estado: Session, Application y Page	5
2.3. Patrones de Arquitectura para Aplicaciones Web	5
2.3.1. Arquitectura por capas (Layered Architecture)	6
2.3.2. Otras arquitecturas destacadas	6
2.4. Diseño de Arquitecturas Web Escalables	6
2.4.1. Patrones de Caché en Aplicaciones Web	7
2.4.2. Documentación y Evaluación	7
3. Desarrollo	7
3.1. Gestión de Estado Stateless y Seguridad	7
3.2. Persistencia, Paginación y Datos Semilla (Flyway)	8
3.3. Patrones de Arquitectura Implementados	8
3.4. Modelado y Diagramas del Sistema	9
3.4.1. Diagrama de Clases UML y ERD	9
3.4.2. Modelo C4: Contexto y Contenedores	9
3.5. Architecture Decision Records (ADR)	11
3.6. Medición de Rendimiento y Speedup	12
3.7. Frontend: Interfaz Gráfica y Consumo de API (Angular)	13
4. Conclusiones	16

1. Introducción

El desarrollo de software empresarial contemporáneo exige sistemas distribuidos, escalables y seguros. El presente informe documenta el desarrollo de la Práctica Experimental de la Unidad 3, donde se integró un stack tecnológico robusto compuesto por Java 21, Spring Boot 3.3, Angular 17+ y PostgreSQL 16 [5].

El objetivo principal radica en aplicar patrones de arquitectura reconocidos en la industria, tales como la arquitectura en capas y el patrón *Cache-Aside* [2], con el fin de optimizar el rendimiento y la mantenibilidad del código. Además, se abordó la seguridad desde un enfoque sin estado (*stateless*), utilizando JSON Web Tokens (JWT) [3] gestionados bajo estrictas políticas de seguridad mediante cookies `HttpOnly`, e implementando un mecanismo de revocación de tokens apoyado en Redis. En este documento se detallan las decisiones arquitectónicas (ADR), el análisis de rendimiento (benchmark) y el modelado del sistema (UML, ERD y C4).

2. Marco Teórico: Fundamentos de Arquitectura y Seguridad Web

2.1. Seguridad en Aplicaciones Web

La seguridad en aplicaciones web es el conjunto de medidas, técnicas y buenas prácticas orientadas a proteger los sitios y sistemas que se ejecutan en un servidor frente a accesos no autorizados, robo de información, interrupción del servicio o manipulación de datos. En un entorno cliente-servidor, la superficie de ataque incluye el propio código, el servidor, la base de datos y la red. Por ello, la seguridad debe abordarse mediante el principio de "seguridad por diseño" (security by design).

2.1.1. Principios fundamentales de la seguridad

La seguridad se apoya tradicionalmente en tres pilares conocidos como la tríada CIA:

- **Confidencialidad:** Garantizar que la información solo sea accesible para quienes están autorizados.
- **Integridad:** Asegurar que los datos no sean alterados de forma no autorizada.

- **Disponibilidad:** Garantizar que el sistema y la información estén accesibles.

A estos se añaden la **Autenticación** (verificar quién es el usuario) y la **Autorización** (controlar qué puede hacer).

2.1.2. OWASP Top 10 y Principales Amenazas

OWASP (Open Worldwide Application Security Project) resume los riesgos más críticos en aplicaciones web:

- **A01 Pérdida de control de acceso:** Aplicar el principio de mínimo privilegio.
- **A02 Fallos criptográficos:** Cifrar datos en tránsito (TLS) y en reposo.
- **A03 Inyección (Injection):** Usar consultas parametrizadas o ORM, y validar entradas.
- **A04 Diseño inseguro:** Incorporar modelado de amenazas desde el diseño.
- **A05 Configuración de seguridad incorrecta:** Aplicar bastionado (hardening).
- **A06 Componentes vulnerables:** Inventariar dependencias y aplicar parches.
- **A07 Fallos de autenticación:** Implementar MFA y políticas de contraseñas.
- **A08 Fallos de integridad de software:** Verificar firmas digitales y CI/CD seguros.
- **A09 Fallos de monitorización:** Centralizar logs y establecer alertas.
- **A10 SSRF (Server-Side Request Forgery):** Limitar URLs usando listas blancas.

2.1.3. Buenas prácticas de codificación segura

- **Validación de entradas:** Validar en el servidor mediante listas blancas y realizar un correcto escapado de salida.
- **Gestión de sesiones:** Usar funciones de hash fuertes (bcrypt, Argon2) para contraseñas. Para cookies, usar atributos `HttpOnly`, `Secure` y `SameSite`.

- **Cifrado:** Forzar HTTPS obligatorio y no incluir secretos o tokens en el código fuente.
- **Mínimo privilegio:** Denegar el acceso por defecto y verificar en cada petición.

2.2. Objetos Integrados y Gestión de Estado en Aplicaciones Web

2.2.1. El objeto Request y Response

- **Request:** Representa la solicitud que envía el cliente al servidor. Contiene los datos enviados por formularios (GET/POST), cabeceras HTTP, cookies y parámetros de URL (query string). Se usa típicamente para leer datos introducidos por el usuario o detectar el tipo de navegador.
- **Response:** Representa la respuesta del servidor hacia el cliente. Permite escribir contenido HTML o JSON, controlar cabeceras, establecer cookies y realizar redirecciones.

2.2.2. Gestión de estado: Session, Application y Page

Dado que HTTP es un protocolo *stateless* (sin estado), se requieren mecanismos adicionales:

- **Session:** Mantiene información específica de un usuario a lo largo de múltiples peticiones mediante un Session ID (generalmente en una cookie). Persiste mientras el usuario navega, hasta expirar por inactividad. Consume memoria del servidor.
- **Application:** Representa el estado global compartido por todos los usuarios. Requiere sincronización (bloqueos) para evitar condiciones de carrera.
- **Page:** Gestiona el ciclo de vida de una página individual (inicialización, carga y descarga).

2.3. Patrones de Arquitectura para Aplicaciones Web

La arquitectura define la organización del sistema, buscando equilibrar requisitos funcionales y atributos de calidad (escalabilidad, mantenibilidad, etc.). Se rige por principios

como la separación de responsabilidades, bajo acoplamiento, alta cohesión, y principios como DRY (Don't Repeat Yourself) y KISS (Keep It Simple).

2.3.1. Arquitectura por capas (Layered Architecture)

Organiza el sistema en niveles horizontales. Las capas típicas son:

- Capa de presentación (Interfaz de usuario).
- Capa de lógica de negocio (Reglas y procesos del sistema).
- Capa de acceso a datos (Comunicación con persistencia).
- Capa de datos (Almacenamiento).

Ventaja: Facilita el mantenimiento y separación de responsabilidades. **Desventaja:** Puede introducir sobrecarga de rendimiento al atravesar varias capas.

2.3.2. Otras arquitecturas destacadas

- **En pizarra (Blackboard):** Componentes acceden a un espacio compartido para leer/escribir de forma colaborativa (ej. IA).
- **Dirigida por eventos (Event-Driven):** Comunicación asíncrona mediante un bus de eventos, ideal para sistemas reactivos e IoT.
- **Peer-to-peer (P2P):** Cada nodo actúa como cliente y servidor simultáneamente, eliminando el punto único de fallo (ej. Blockchain).
- **SOA y Microservicios:** SOA organiza servicios con protocolos estándar y ESB. Microservicios propone dividir la aplicación en servicios muy pequeños e independientes, escalables por separado y con base de datos propia, aunque incrementan la complejidad operativa.

2.4. Diseño de Arquitecturas Web Escalables

La escalabilidad es la capacidad de manejar un aumento en la carga sin degradar el rendimiento.

- **Escalabilidad vertical (Scale up):** Aumentar la capacidad del mismo servidor (CPU, RAM). Tiene un límite físico.
- **Escalabilidad horizontal (Scale out):** Añadir más servidores que trabajen en paralelo. Requiere balanceadores de carga y diseño de servicios sin estado (*stateless*).

2.4.1. Patrones de Caché en Aplicaciones Web

El caché almacena temporalmente resultados frecuentes para reducir la latencia del backend.

- **Estrategias:** *Cache-aside* (lazy loading), *Write-through* (escritura síncrona), *Write-back* (escritura asíncrona).
- **Consideraciones:** Mejora el rendimiento pero introduce problemas de consistencia de datos. Requiere definir bien las políticas de expiración (TTL) y reemplazo (LRU/LFU).

2.4.2. Documentación y Evaluación

Una arquitectura debe documentarse (vistas 4+1, UML, y ADRs) y evaluarse (ej. mediante ATAM) para justificar decisiones y mitigar riesgos asociados a los atributos de calidad (rendimiento, seguridad, mantenibilidad).

3. Desarrollo

3.1. Gestión de Estado Stateless y Seguridad

Para la seguridad de la aplicación, se prescindió de las sesiones tradicionales en servidor en favor de un enfoque *stateless* basado en JWT. Para prevenir ataques de tipo Cross-Site Scripting (XSS), el token JWT nunca se expone al entorno de JavaScript del cliente (Angular), sino que el backend lo adjunta como una cookie `HttpOnly` con el atributo `SameSite=Lax` tras un inicio de sesión exitoso.

Para garantizar que el proceso de *logout* invalide efectivamente el token antes de su fecha de expiración, se implementó una **lista negra (blacklist) en Redis**. Al cerrar sesión, el ID único del token (JTI) se registra en Redis con un tiempo de vida (TTL) igual

al tiempo restante de validez del JWT. El filtro de seguridad de Spring (`JwtAuthFilter`) verifica contra Redis en cada petición; si el JTI está en la lista negra, la petición es rechazada (403 Forbidden), demostrando una revocación total y efectiva.

3.2. Persistencia, Paginación y Datos Semilla (Flyway)

La persistencia de datos relacionales se gestionó mediante PostgreSQL 16 y Spring Data JPA [5]. Se construyó un CRUD completo para la entidad Vehículo. Para optimizar el tráfico de red y la memoria, las consultas al listado se implementaron empleando la interfaz `Pageable`, permitiendo una paginación dinámica controlada desde el cliente Angular.

Para asegurar un entorno de pruebas consistente, se integró la herramienta de migración de bases de datos **Flyway**. Mediante el script `V3__datos_semilla.sql`, se automatizó la carga inicial de un mínimo de 50 registros de prueba en la tabla de vehículos al arrancar el contenedor, facilitando las pruebas de paginación y caché sin intervención manual.

3.3. Patrones de Arquitectura Implementados

El proyecto incorpora dos patrones fundamentales descritos en la literatura académica [2]:

- **Arquitectura en Capas:** El código backend se segmentó estrictamente en Controller (exposición API REST), Service (lógica de negocio y caché), Repository (acceso a datos) y Entity (modelado del dominio).
- **Patrón Cache-Aside:** Se integró Redis como motor de caché para las consultas de lectura [4]. Mediante la anotación `@Cacheable`, la aplicación consulta primero a Redis; en caso de *miss*, consulta a PostgreSQL y guarda el resultado en caché. Las operaciones de escritura y eliminación utilizan `@CacheEvict` para invalidar los registros modificados, manteniendo la consistencia de los datos.

3.4. Modelado y Diagramas del Sistema

3.4.1. Diagrama de Clases UML y ERD

A continuación, se presenta el diagrama de clases UML refactorizado, abarcando todos los paquetes implementados. Seguidamente, se incluye el Diagrama Entidad-Relación (ERD) exportado desde pgAdmin 4.

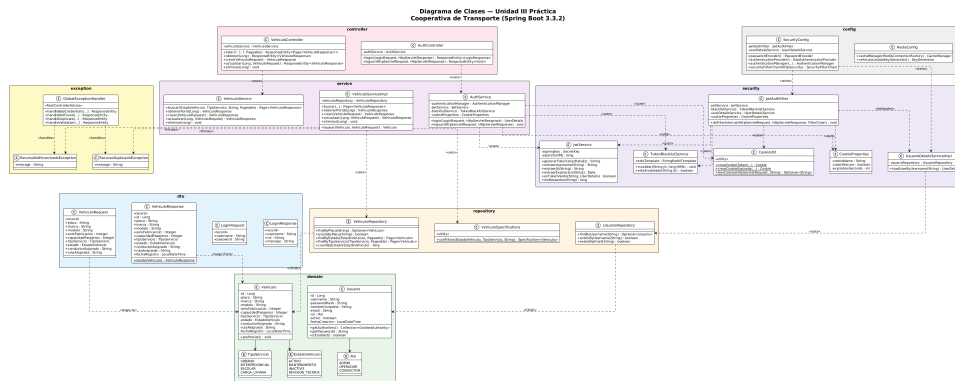


Figura 1: Diagrama de Clases UML del sistema backend.

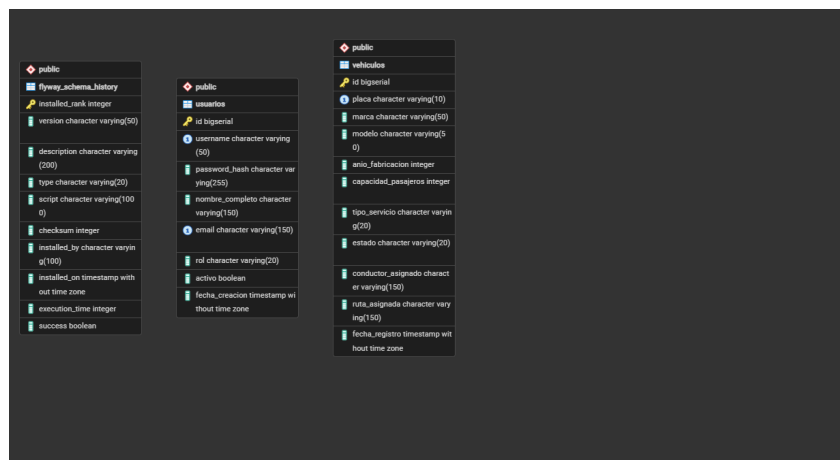


Figura 2: Diagrama Entidad-Relación (ERD).

3.4.2. Modelo C4: Contexto y Contenedores

Para documentar la arquitectura a un nivel superior, se aplicó el Modelo C4 [1]. El Nivel 1 muestra las interacciones del sistema con los usuarios externos, mientras que el Nivel 2 desglosa las piezas de software desplegadas en los contenedores Docker.

Nivel 1 — Diagrama de Contexto (C4) Cooperativa de Transporte - Unidad III

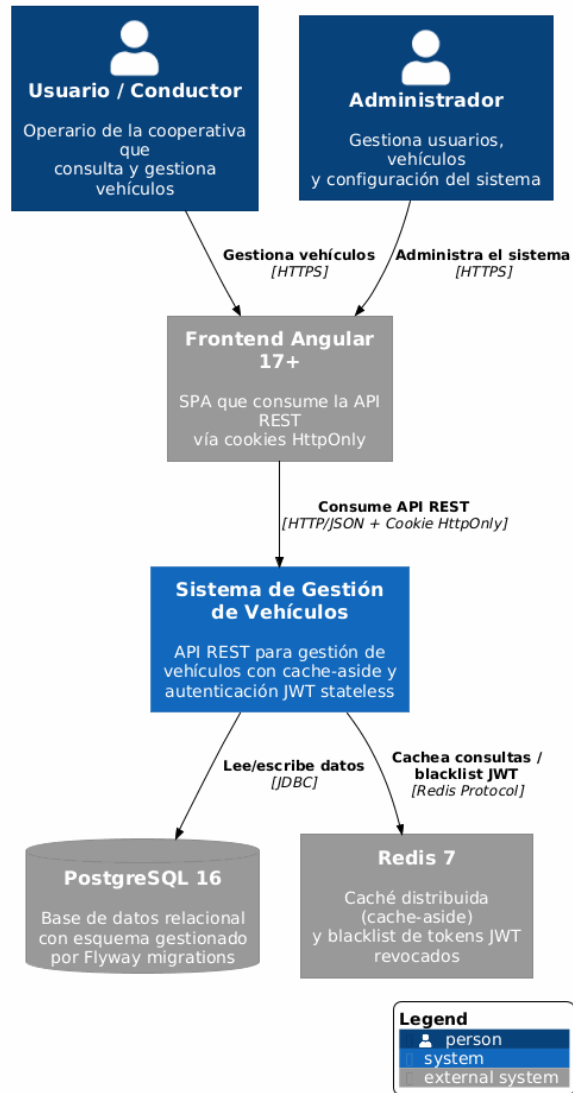


Figura 3: Modelo C4: Nivel 1 - Contexto del Sistema.

Nivel 2 — Diagrama de Contenedores (C4) Cooperativa de Transporte - Unidad III

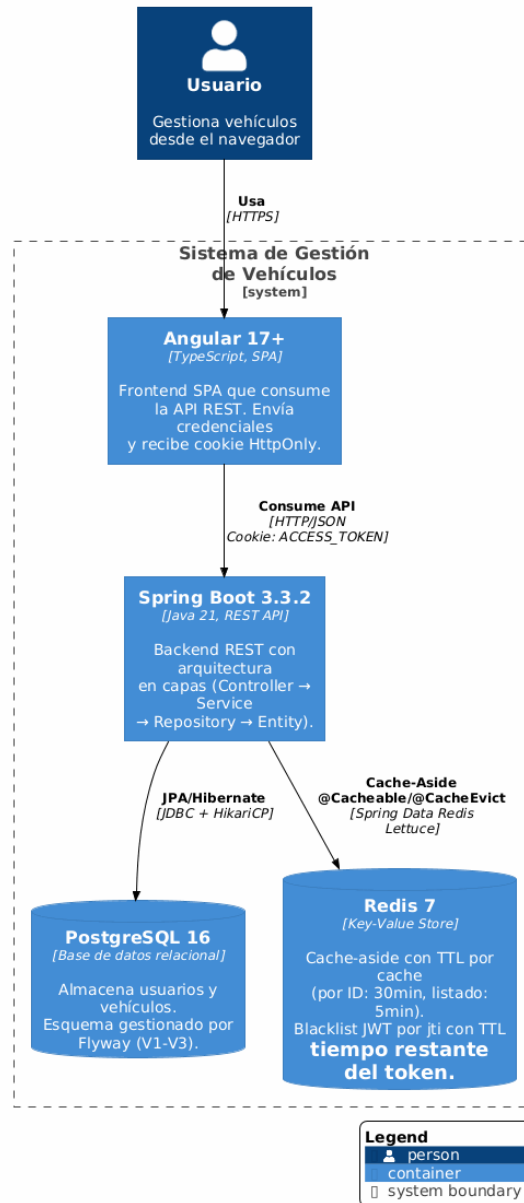


Figura 4: Modelo C4: Nivel 2 - Contenedores.

3.5. Architecture Decision Records (ADR)

ADR-002: Elección de Angular vs. React para el Frontend

Estado: Aceptado

Contexto: Se requería un framework o librería para consumir la API REST, con necesidades fuertes de tipado, inyección de dependencias y estructura corporativa.

Decisión: Se seleccionó Angular 17+.

Consecuencias: Angular proporciona TypeScript nativo, interceptores HTTP incorporados y un sistema de enrutamiento integrado, lo cual acelera el desarrollo seguro (por ejemplo, enviando el flag `withCredentials: true` nativamente). La curva de aprendizaje es más pronunciada frente a React.

Alternativas consideradas: React JS, descartado por requerir librerías de terceros adicionales para el enrutamiento y manejo de estado complejo.

ADR-003: Elección de Redis para Caché y Blacklist JWT

Estado: Aceptado

Contexto: El listado de vehículos paginado sufría de latencia de I/O de disco en consultas recurrentes. Además, al usar JWT *stateless*, se necesitaba invalidar tokens tras un logout sin almacenar sesiones persistentes.

Decisión: Utilizar Redis 7 para ambos propósitos (Patrón Cache-Aside y Blacklist).

Consecuencias: Disminución radical del tiempo de respuesta (speedup). Se requiere mantener un contenedor Docker adicional y controlar los tiempos de vida (TTL) estrictamente.

Alternativas consideradas: Caché en memoria (ConcurrentHashMap), descartada por no ser escalable horizontalmente en entornos distribuidos.

3.6. Medición de Rendimiento y Speedup

Se ejecutaron pruebas de *benchmark* invocando la consulta principal del listado de vehículos 10 veces, midiendo el tiempo de respuesta con caché de Redis (T_{con}) y sin caché (T_{sin}). Los resultados fueron extraídos de los archivos de prueba generados (`con_cache.csv` y `sin_cache.csv`).

Tabla 1: Tiempos de respuesta (segundos) y Speedup.

Métrica	Sin Caché (T_{sin})	Con Caché (T_{con})
Promedio	0.034130 s	0.019998 s
P95	0.040664 s	0.024305 s

Análisis de mejora: Utilizando la fórmula $S = \frac{T_{sin}}{T_{con}}$, obtenemos:

$$S = \frac{0,034130}{0,019998} \approx 1,71$$

El sistema logró un **speedup de 1.71x**. Esto indica que las respuestas desde memoria (Redis) son un 71 % más rápidas en promedio que las consultas directas con persistencia en disco a través del ORM (PostgreSQL).

3.7. Frontend: Interfaz Gráfica y Consumo de API (Angular)

El cliente web fue desarrollado para proporcionar una experiencia de usuario (UX) altamente interactiva y responsiva. El sistema maneja de forma segura las transiciones de estado, garantizando que solo los usuarios autenticados puedan acceder a los recursos protegidos. A continuación, se detallan las diferentes vistas y estados del sistema:

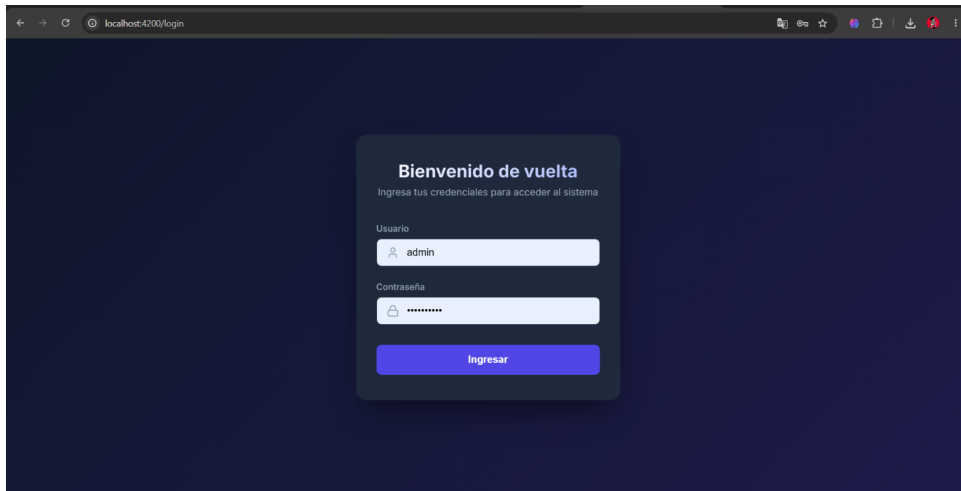


Figura 5: Interfaz de Inicio de Sesión rediseñada (Premium UI).

La pantalla de inicio de sesión (**Interfaz-Inicio.png**) emplea formularios reactivos (*Reactive Forms*) para validar las credenciales en tiempo real. Utiliza una estética moderna basada en el diseño "Glassmorphism", que además de mejorar el aspecto visual, intercepta las respuestas HTTP para gestionar los tokens JWT de manera transparente.

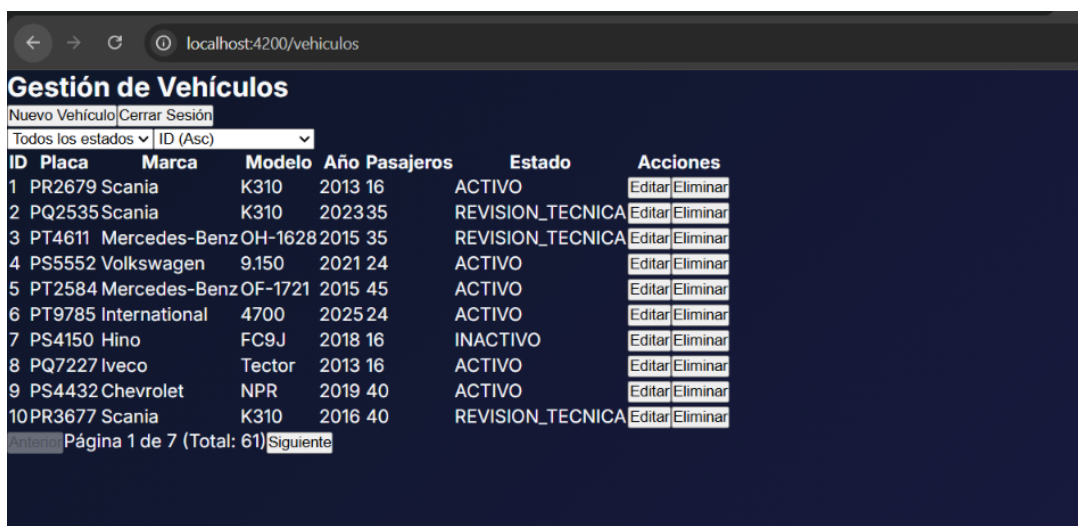


Figura 6: Panel principal de gestión de vehículos (CRUD) con Paginación.

Una vez autenticado el usuario, el sistema redirige al panel principal (**Interfaz-Vehiculos.png**). En esta vista se encuentra la tabla de datos que consume directamente la API paginada de Spring Boot. Esta tabla se alimenta de la caché de Redis en el backend, lo que permite una carga casi instantánea de la información, optimizando significativamente la experiencia de navegación.

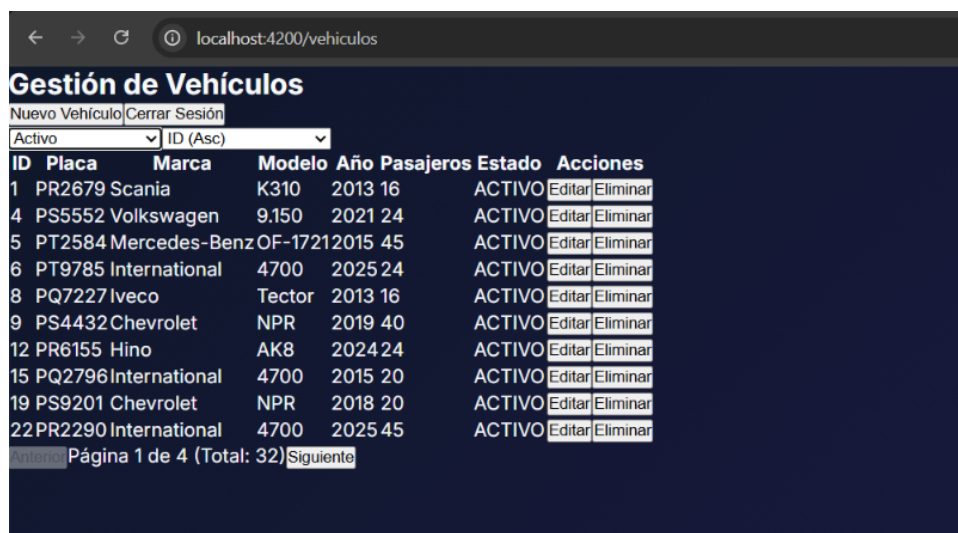


Figura 7: Gestión de Vehículos filtrados por Estado: Activo.

El sistema ofrece filtros dinámicos que se aplican sobre la base de datos a través de peticiones optimizadas. En la figura superior (**Interfaz-Vehiculos-Activo.png**) se visualiza la respuesta del sistema al solicitar exclusivamente el inventario operativo, destacando visualmente las insignias de estado para un rápido reconocimiento.



ID	Placa	Marca	Modelo	Año	Pasajeros	Estado	Acciones
25	PT2982	Volkswagen	17190	2022	40	MANTENIMIENTO	Editar Eliminar
34	PT1960	Mercedes-Benz	OH-1628	2014	30	MANTENIMIENTO	Editar Eliminar
39	PR1651	Isuzu	FTR	2013	40	MANTENIMIENTO	Editar Eliminar
41	PQ1152	Chevrolet	NPR	2018	20	MANTENIMIENTO	Editar Eliminar
50	PS6974	Hino	AK8	2025	24	MANTENIMIENTO	Editar Eliminar
54	PT4644	Mercedes-Benz	OF-1721	2024	40	MANTENIMIENTO	Editar Eliminar

Figura 8: Gestión de Vehículos filtrados por Estado: En Mantenimiento.

Al cambiar los parámetros de búsqueda, el frontend actualiza la interfaz sin recargar la página. La vista (**Interfaz-Vehiculos-Mantenimiento.png**) demuestra la flexibilidad del componente para renderizar estados críticos de la flota de transporte, permitiendo al administrador tomar decisiones operativas.



ID	Placa	Marca	Modelo	Año	Pasajeros	Estado	Acciones
14	PT7543	Chevrolet	NPR	2014	20	INACTIVO	Editar Eliminar
7	PS4150	Hino	FC9J	2018	16	INACTIVO	Editar Eliminar
37	PQ1821	Hino	AK8	2021	40	INACTIVO	Editar Eliminar
48	PR8239	Hino	AK8	2017	45	INACTIVO	Editar Eliminar
51	PS7658	Hino	AK8	2017	20	INACTIVO	Editar Eliminar
30	PQ2612	International	4700	2018	30	INACTIVO	Editar Eliminar
57	PS1452	International	4700	2022	35	INACTIVO	Editar Eliminar
60	PQ9518	Isuzu	FTR	2015	24	INACTIVO	Editar Eliminar
44	PS2697	Iveco	Tector	2016	35	INACTIVO	Editar Eliminar
20	PR3504	Mercedes-Benz	OF-1721	2020	16	INACTIVO	Editar Eliminar

Página 1 de 2 (Total: 15) [Siguiende](#)

Figura 9: Gestión de Vehículos filtrados por Estado: Inactivo.

Finalmente, la vista para los vehículos fuera de servicio (**Interfaz-Vehiculos-Inactivo.png**) corrobora el funcionamiento end-to-end del filtrado múltiple. Las distintas consultas a la API demuestran cómo la estructura `Pageable` de Spring Data resuelve eficientemente las combinaciones de estado y paginación directamente desde el Frontend de Angular.

4. Conclusiones

- La adopción del ecosistema Java 21 con Spring Boot, sumado a una correcta orquestación con Docker y Flyway para la migración de la semilla de 50 registros, proporcionó un entorno de desarrollo reproducible, ágil y escalable.
- Se demostró con éxito que el uso de JWT enviados mediante Cookies HttpOnly, combinados con una lista negra centralizada en Redis, mitiga las vulnerabilidades del lado del cliente (XSS) a la vez que provee control total para la invalidación de sesiones en esquemas *stateless*.
- La inclusión de Redis como capa de caché (Cache-Aside) es altamente efectiva. Como se demostró empíricamente, se liberó carga transaccional de PostgreSQL y se aceleró el tiempo de respuesta del listado paginado en un factor de 1.71x.
- La documentación a través del Modelo C4, Diagramas UML y ADRs consolida la práctica ingenieril del proyecto, alineando la implementación técnica de Angular y Spring Data con metodologías estandarizadas de la industria del software.

Referencias

- [1] Brown, S. (2018). *Software Architecture for Developers: Visualise, document and explore your software architecture*. Leanpub.
- [2] Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
- [3] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force (IETF). Recuperado de <https://datatracker.ietf.org/doc/html/rfc7519>
- [4] Redis Ltd. (2024). *Caching with Redis - Best Practices and Cache-Aside Pattern*. Recuperado de <https://redis.io/docs/manual/patterns/>
- [5] Walls, C. (2022). *Spring in Action* (6ta ed.). Manning Publications.
- [6] Mozilla. (s. f.). *Web Security Guidelines*. Mozilla Infosec. Recuperado de https://infosec.mozilla.org/guidelines/web_security
- [7] National Institute of Standards and Technology. (2020). *Digital Identity Guidelines: Authentication and Lifecycle Management (NIST SP 800-63B)*. U.S. Department of Commerce. Recuperado de <https://csrc.nist.gov/pubs/sp/800/63/b/upd2/final>
- [8] OWASP Foundation. (2021). *OWASP Top 10:2021 — The ten most critical web application security risks*. Recuperado de <https://owasp.org/Top10/2021/>
- [9] OWASP Foundation. (s. f.). *OWASP Cheat Sheet Series*. Recuperado de <https://cheatsheetseries.owasp.org/>
- [10] Parlamento Europeo y Consejo de la Unión Europea. (2016). *Reglamento (UE) 2016/679, de 27 de abril de 2016, relativo a la protección de las personas físicas (RGPD)*. Diario Oficial de la Unión Europea, L 119.
- [11] Stuttard, D., & Pinto, M. (2011). *The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws* (2.^a ed.). Wiley.

- [12] Richards, M. (2022). *Software Architecture Patterns* (2nd ed.). O'Reilly Media. Recuperado de <https://www.oreilly.com/library/view/software-architecture-patterns/9781098134280/ch03.html>
- [13] Microsoft Learn / Azure Architecture Center. (s. f.). *Cache-Aside Pattern*. Recuperado de [https://learn.microsoft.com/en-us/previous-versions/msp-n-p/dn589799\(v=pandp.10\)](https://learn.microsoft.com/en-us/previous-versions/msp-n-p/dn589799(v=pandp.10))
- [14] Software Engineering Institute, Carnegie Mellon University. (s. f.). *The Architecture Tradeoff Analysis Method*. Recuperado de <https://www.sei.cmu.edu/library/the-architecture-tradeoff-analysis-method-2>